# Announcements

- Assignment 1
  - Connecting *n* Linux filters with pipes
  - Due 11:59 p.m., Wednesday, January 28
- Intercollegiate Programming Contest
  - 30th Annual Conference of the Consortium for Computing Sciences in Colleges Northeast Region (CCSCNE), April 10-11, 2026.
  - 9 a.m. – Noon, April 10
  - Team of three
  - Interested? Email me by this Friday.

CPSC 375 High-Performance Computing

Lecture 3

# Named Pipes

# Linux IPC

- Pipes
- Message Queues
- Shared memory
- Semaphores
- Signals
- Sockets

# Linux Pipes

- *pipe*: a circular buffer of fixed size written by one process and read by another
- Allows anonymous (unnamed) communication
- Exists between related processes (sharing a common ancestor)
- Unidirectional
- Producer / Consumer

# Linux Pipes

- Created by `pipe()` system call
- Data pass through file by `write()` and `read()` system calls
- Data stored in kernel in virtual filesystem
- OS enforces mutual exclusion: only one process at a time can access the pipe.

# Named Pipes (FIFOs)

- Variant to pipes that allows *unrelated* processes to communicate
- Created with
  - `mkfifo()` library function (sits on top of `mknod()` system call)
  - `mkfifo` shell command
- Exist as a regular file
- Read and written like regular file
- Data is stored in kernel
- Must be open at both ends before reading or writing
- OS enforces mutual exclusion

# Example

```
// Writer
mkfifo("myfifo", 0666);
int fdw = open("myfifo", O_WRONLY);
write(fdw, "CPSC 375", 9);
close(fdw);
```

```
// Reader
char buffer[100];
int fdr = open("myfifo", O_RDONLY);
read(fdr, buffer, 100);
printf("Data received: %s\n", buffer);
close(fdr);
```

# Creating FIFOs Using Command Line

```
$ mkfifo mypipe
$ ls -l mypipe
prw------ 1 pyoon pyoon 0 Jan 25 22:44 mypipe|
```

# Limits on Pipes and FIFOs

- Atomicity:
  - I/O is *atomic* for data quantities less than PIPE_BUF
  - Typical: PIPE_BUF = 4096 bytes
- Write capacity:
  - Typically 64K
  - Writers block or fail if pipe is full

# Blocking vs Polling

- By default, pipes are in *blocking mode*
- Process suspends its execution and waits for the kernel to signal that the operation can be completed.
  - Process attempts to read() from an empty piped or
  - Process attempts to write() to a full pipe

```
// Reader (blocking)
int fdr = open("myfifo", O_RDONLY);
char buf[100];
read(fdr, buf, 100); // Process suspends if FIFO is empty
printf("Data received: %s\n", buf);
```

# Blocking vs Polling, cont'd

- *Polling* occurs when a process uses *non-blocking I/O*.
  - Enabled by passing the O_NONBLOCK flag when opening a FIFO
  - read() or write() call returns immediately
  - If the pipe is empty (for a reader) or full (for a writer), the system call returns -1
  - Process typically enters a "polling loop" where it repeatedly checks the status of the pipe

# Blocking vs Polling, cont'd

```
// Reader (polling)
int fdr = open("myfifo", O_RDONLY | O_NONBLOCK);
char buf[100];
while (1) {
    int n = read(fdr, buf, 100);
    if (n > 0) {
        printf("Data received: %s\n", buf);
        break;
    } else {
        do_something_useful();
        printf("still waiting...\n");
        sleep(1); // optional
    }
}
```

# Multiple Readers and Writers

- Both pipes and FIFOs support multiple readers and writers
- Multiple readers
  - When multiple processes read from the same FIFO, they read in a first-come, first-served order.
  - Data removed from the kernel buffer but not broadcast to all readers.
- Multiple writers
  - Kernel must decide how to interleave the data.
  - Writes smaller than PIPE_BUF, the kernel guarantees the write is atomic.
  - Writes longer than PIPE_BUF from multiple writers may be interleaved.
- Must use synchronization if read/write order must be guaranteed

# FIFOs Are NOT Files

```
// file
int fdw = open("myfile", O_WRONLY);
write(fdw, "CPSC 375", 9);
close(fdw);
```

```
// pipe
mkfifo("myfifo", 0666);
int fdw = open("myfifo", O_WRONLY);
write(fdw, "CPSC 375", 9);
close(fdw);
```

- Both files FIFOs must be opened for reading and writing before either may occur
- Files exist in filesystem
- Files have no atomicity guarantee
- Files have no practical capacity limit